I, OWASP WSTG DEFINITION

OWASP is a globally recognized, open-source, and non-profit foundation dedicated to improving the security of software. Operating as a collaborative community of security experts, developers, and researchers from around the world, OWASP produces freely available articles, methodologies, documentation, and tools. By fostering an open exchange of knowledge, the foundation has established itself as an authoritative voice in application security, widely known for foundational resources such as the OWASP Top 10, which outlines the most critical web application security risks.

Among OWASP's most comprehensive and essential contributions to the field is the Web Security Testing Guide (WSTG). The WSTG is a premier cybersecurity testing resource designed specifically for web application developers and security professionals. Rather than serving merely as a checklist of known flaws, the WSTG provides a rigorous, peer-reviewed, and systematic framework that details precisely how to test for a vast array of web vulnerabilities. It represents the culmination of years of collaborative industry effort, consolidating best practices into a unified methodology that can be consistently applied across different organizations and technologies.

The primary objective of the WSTG is to provide an exhaustive roadmap for penetration testing and vulnerability assessment. It guides testers through the entire security testing lifecycle, beginning with initial information gathering and configuration management, and progressing through deep, category-specific testing phases—such as identity and authentication management, authorization, input validation, and business logic testing. Crucially, the WSTG emphasizes the necessity of human-led intelligence in the testing process. While it acknowledges the utility of automated scanning tools, the guide is explicitly structured to uncover the complex, context-dependent vulnerabilities—particularly within application logic and access controls—that automated scanners consistently fail to detect.

II, OWASP TESTING STRUCTURE

The WSTG structure included :

+ The testing frameworks:Indicates "when" to do the security testing and how to links security testing directly to the Software Development Life Cycle (SDLC).

+ The web application security testing: evaluating the security of a computer system or network by methodically validating and verifying the effectiveness of application security controls through vulnerability assessments. In the security testing framework , there will be 2 categories: Passive testing and Active testing. The technical core of the WSTG, containing actionable test cases split across 12 specific operational domains.

1, The testing framework

The Testing Framework presents a strategic blueprint for shifting security away from a purely reactive, "point-in-time" evaluation model and integrating it holistically across the entire Software Development Life Cycle (SDLC).

The core thesis of this framework is that relying entirely on traditional penetration testing at the end of a deployment cycle is a costly and inefficient strategy. Instead, an organization must measure and verify security continuously—evaluating its people, processes, and technology across five distinct development phases. This report breaks down the underlying philosophy, the step-by-step phases, the typical testing workflow, and how this methodology interfaces with external standards.

The guide builds a foundation on what an effective testing program must address. Software security is defined as a combination of three interlinked pillars:

+ People: Testing education, awareness, and skill levels to prevent bugs at the point of origin.

+ Process: Ensuring clear security policies, robust standards, and documented workflows exist—and that teams know how to execute them.

+ Technology: Inspecting the actual code, configurations, and running systems to verify the effectiveness of the people and processes.

The framework is explicitly designed to be non-prescriptive. It serves as a flexible reference model that companies can customize to fit their unique organizational culture, development speed (e.g., DevOps, Agile, Waterfall), and technical stacks.

The SDLC reference framework are divided into 5 phase:

Phase 1: Before Development Begins

├── 1.1 Define a Secure SDLC: Establish a formalized development lifecycle where security check-gates are native to each milestone.

├── 1.2 Review Policies & Coding Standards: Ensure appropriate guardrails exist, such as secure coding standards (e.g., language-specific guidelines for Java, .NET, or Go) and cryptographic rules. Documenting predictable patterns reduces arbitrary design decisions later.

└── 1.3 Develop Measurement, Metrics & Traceability: Design a measurement program *before* writing code. This outlines how security defects will be classified and traced, allowing the business to measure process efficiency and software quality over time.

Phase 2: During Definition and Design

├── 2.1 Review Security Requirements (Test Assumptions & Gaps): Test the requirements definitions to identify omissions and challenge underlying assumptions. Reviews focus

extensively on fundamental security mechanisms: authentication, authorization, session management, data confidentiality, and legislative compliance

├── 2.2 Review Design & Architecture (Centralize Controls): Validate structural architectures. This is the optimal time to enforce central, shared security utilities (e.g., a centralized data validation or uniform authorization framework) rather than writing disconnected controls in hundreds of individual modules

├── 2.3 Create & Review UML Models (Logic & Flows): Build and analyze Unified Modeling Language models to align the security team and system architects on exactly how data and logic will flow through the application.

└── 2.4 Create & Review Threat Models (Identify Risks): Execute realistic threat scenario simulations. Determine if identified vectors are mitigated by the architecture, accepted via documented risk thresholds, or transferred appropriately

Phase 3: During Development

└── Code Review, Walk-throughs, and Static Analysis (SAST): High-level conceptual sessions where developers present the logic flow of implemented modules directly to the security team, justifying technical design choices.

Phase 4: During Deployment

├── 4.1 Application Penetration Testing (Dynamic/DAST Validation): Operating under the assumption that some flaws always bypass early gates, dynamic testing verifies the fully integrated software at runtime against real-world attack vectors.

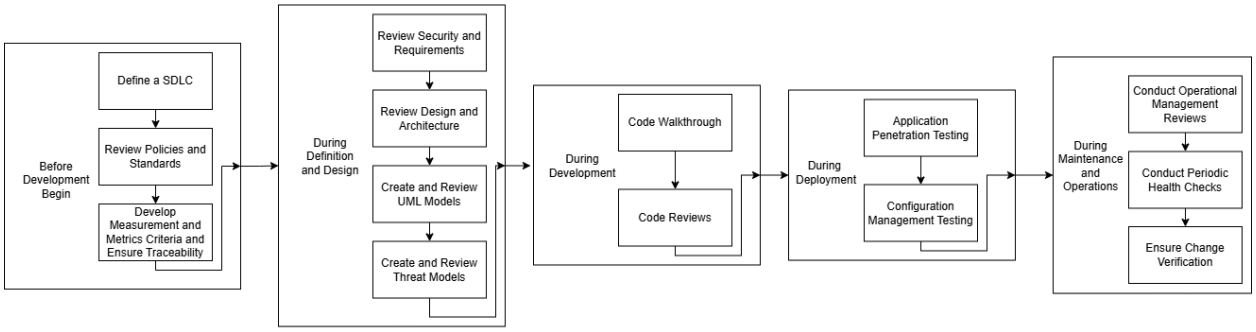
└── 4.2 Configuration Management Testing (Infrastructure Hardening): Focuses heavily on the underlying infrastructure, server platforms, and cloud environments. It verifies that default vendor accounts/files are removed, debugging modules are deactivated, custom error handling is enabled (preventing stack-trace leaks), and server processes run under the principle of least privilege.

Phase 5: During Maintenance and Operations

├── 5.1 Conduct Operational Management Reviews: Audit ongoing data handling workflows, log aggregation systems, and administrative access controls.

├── 5.2 Conduct Periodic Health Checks (Regression & Vulnerability Scans): Execute monthly or quarterly recurring security checks to identify regression bugs or public vulnerabilities (CVEs) emerging in third-party dependencies.

└── 5.3 Ensure Change Verification (Patch & Delta Testing): Embed delta-security testing directly into the change management process. Every minor update, patch, or feature addition must pass through a validation gate before transitioning from QA into production.



2, Web application testing guide

- This section provides the specific, highly technical blueprints required to execute security assessments. It defines a structured, comprehensive, and repeatable methodology consisting of 12 testing domains broken down into dozens of individual test cases. Each test case is assigned a unique identifier and provides security practitioners, QA engineers, and developers with clear objectives, step-by-step validation techniques, and remediation guidance.

- To ensure consistency across global testing teams, the testing guide standardizes the layout of every technical test case. Each guide entry contains the following mandatory subsections:

+ Identifier Code: Follows the convention WSTG-[DOMAIN]-[NUMBER] (e.g., WSTG-INP-05 for SQL Injection).

+ Summary: A concise explanation of the vulnerability class, its underlying technical causes, and the associated risks.

+ Test Objectives: Clear, measurable conditions that the tester must satisfy to verify whether the vulnerability exists.

+ How to Test: The core technical walkthrough, typically divided into:

    + Passive Testing: Analyzing traffic, code, or metadata without actively exploiting or mutating state.

    + Active Testing: Sending payloads, manipulating requests, and forcing the application into unexpected runtime states.

+ Remediation: Actionable, language-agnostic engineering advice on how to patch the root cause of the bug.

+ Tools: A curated list of open-source and commercial utilities (e.g., interception proxies, fuzzers, scanners) tailored to that specific test.

- The Web security testing guide consists of 12 section:

Web Security testing Guide

├── 1. Information Gathering

├── 2. Configuration and Deployment Management Testing

├── 3. Identity Management Testing

├── 4. Authentication Testing

├── 5. Authorization Testing

├── 6. Session Management Testing

├── 7. Input Validation Testing

├── 8. Testing for Error Handling

├── 9. Testing for weak Cryptography

├── 10. Business Logic testing

├── 11. Client-Side testing

└── 12 API testing

1 Information Gathering (INFO)

Reconnaissance is the prerequisite for any successful assessment. This domain focuses on mapping the visible and hidden attack surfaces of the application before launching active attacks.

Key Focus Areas: Fingerprinting web servers and frameworks; reviewing search engine caches and public metafiles (robots.txt, security.txt, sitemap.xml); enumerating applications co-hosted on the infrastructure; mapping out every application entry point and structural execution path.

Objective: To build an exact architectural map of the application, identifying legacy components, unlinked pages, and administrative entry points that might otherwise go unnoticed.

4.2 Configuration and Deployment Management Testing (CONFIG)

Even perfectly written code can be compromised if deployed on a poorly hardened server or cloud environment. This domain targets the platform layer.

Key Focus Areas: Evaluating network infrastructure configuration; checking file extension handling (e.g., ensuring .bak or .old files aren't exposed); enumerating administrative interfaces;

testing permitted HTTP methods (ensuring unsafe options like PUT or DELETE are blocked); validating HTTP Strict Transport Security (HSTS) and cloud storage buckets (e.g., AWS S3 permissions).

Objective: To verify that the application platform is fully hardened, redundant data or debugging endpoints are stripped out, and the environment operates under the principle of least privilege.

4.3 Identity Management Testing (IDNT)

Identity security establishes who a user is within the system. This section examines how user accounts are provisioned, assigned roles, and managed throughout their lifecycle.

Key Focus Areas: Testing horizontal and vertical role definitions; auditing the security of user registration and account self-provisioning processes; checking for account enumeration risks (determining if an attacker can guess valid usernames based on server responses).

Objective: To ensure that authorization boundaries cannot be manipulated at the account creation stage and that attackers cannot guess user lists via system discrepancies.

4.4 Authentication Testing (AUTH)

Authentication verifies the identity claims made during the identity management phase. This domain focuses heavily on credential security.

Key Focus Areas: Testing for weak or unenforced password complexities; checking for default vendor credentials; evaluating account lockout thresholds under brute-force conditions; auditing "remember password" or password-reset logic; verifying that credentials are encrypted in transit and not leaked into browser caches.

Objective: To prevent unauthorized users from gaining access to legitimate accounts through credential stuffing, brute forcing, or logic bypasses.

4.5 Authorization Testing (AZN)

Once authenticated, authorization determines what resources a user is permitted to access. Flaws here break horizontal or vertical isolation boundaries.

Key Focus Areas: Testing for Insecure Direct Object References (IDOR), where altering an ID parameter gives access to another user's data; detecting path/directory traversal (forcing the server to read local system files); verifying vertical privilege escalation (a low-privilege user executing administrative tasks).

Objective: To ensure that users can never access, modify, or delete data outside their explicitly defined permission scope.

4.6 Session Management Testing (SESS)

After authentication, a session token represents the user's ongoing identity. This domain evaluates how securely that state token is generated, stored, and destroyed.

Key Focus Areas: Checking session cookie attributes (Secure, HttpOnly, SameSite); testing for session fixation (where an application allows a pre-existing token to remain valid after login); verifying proper implementation of Cross-Site Request Forgery (CSRF) protections; testing session timeouts and logout finality.

Objective: To ensure session tokens are highly random, robustly protected against interception or client-side theft, and definitively invalidated upon termination.

4.7 Input Validation Testing (INP)

This is historically the most expansive and technical domain in the WSTG. It addresses vulnerabilities that occur when an application trusts user input without sufficient sanitization, filtering, or parameterization.

Key Focus Areas: Detailed execution steps for tracking down a massive matrix of injection attacks: Reflected/Stored Cross-Site Scripting (XSS), SQL Injection (tailored sub-guides for Oracle, MySQL, SQL Server, PostgreSQL, and NoSQL), OS Command Injection, Local and Remote File Inclusion (LFI/RFI), Server-Side Request Forgery (SSRF), and Server-Side Template Injection (SSTI).

Objective: To verify that all untrusted boundaries treat input strictly as data rather than executable commands or structural syntax.

4.8 Testing for Error Handling (ERR)

Applications inevitably fail due to runtime exceptions. This domain monitors how gracefully those failures are processed.

Key Focus Areas: Identifying verbose stack traces; checking for database connection strings, paths, or code logic exposed inside custom error pages.

Objective: To ensure that system failures do not leak structural intelligence that attackers can use to optimize more targeted exploits.

4.9 Testing for Weak Cryptography (CRYP)

Cryptography protects data in transit and at rest. This domain looks for flaws in implementation rather than the math itself.

Key Focus Areas: Auditing Transport Layer Security (TLS) configurations for deprecated protocols (e.g., SSLv3, TLS 1.0); testing for padding oracle vulnerabilities; verifying that sensitive information is never transmitted over plaintext or weak cipher suites.

Objective: To guarantee that data cannot be decrypted or tampered with by an adversary sitting in a Man-in-the-Middle (MitM) network position.

4.10 Business Logic Testing (BUS)

Automated vulnerability scanners generally fail in this category because logic flaws do not trigger traditional syntax errors. This domain requires structural human thinking to abuse the system's intended workflows.

Key Focus Areas: Bypassing standard sequence flows (e.g., skipping a payment step in an e-commerce checkout loop); forge-testing process timing; checking transaction limits (e.g., submitting negative currency amounts); bypassing anti-automation limits; testing malicious and unexpected file uploads.

Objective: To ensure that an application cannot be forced into unauthorized states by an operator executing otherwise "valid" steps out of order or outside acceptable parameters.

4.11 Client-side Testing (CLNT)

Modern web applications execute substantial amounts of code directly inside the user's browser via JavaScript. This domain shifts focus from the server to the client environment.

Key Focus Areas: Investigating DOM-Based XSS; evaluating Cross-Origin Resource Sharing (CORS) configurations; testing WebSockets and Cross-Document Web Messaging security; auditing browser storage mechanisms (localStorage, sessionStorage); checking vulnerability to Clickjacking.

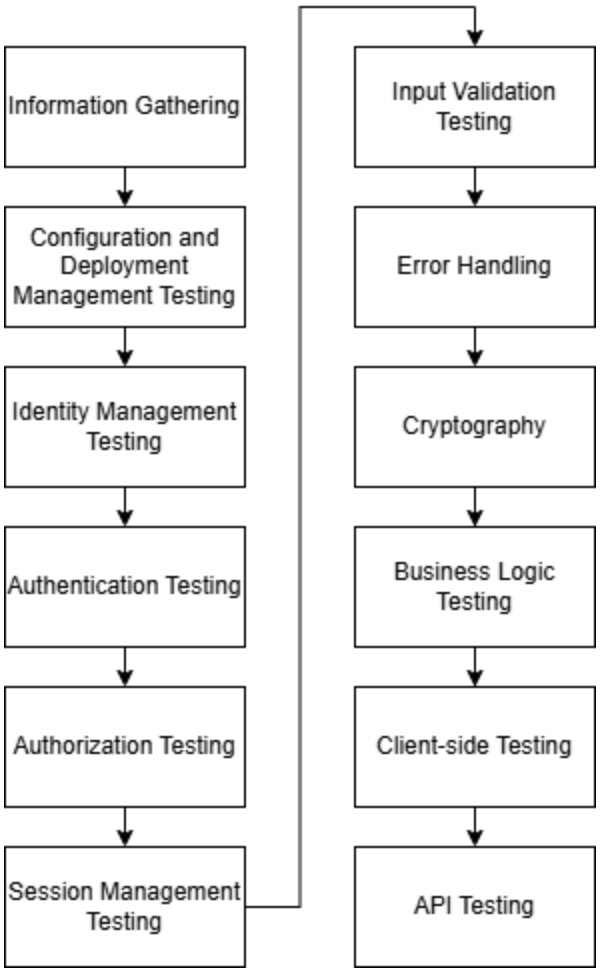
Objective: To protect the end-user from browser-level manipulation that could result in session hijacking, UI redressing, or cross-origin data exposure.

4.12 API Testing (API)

Acknowledging the shift toward decoupled architectures, the guide provides dedicated guidance for modern endpoint interfaces.

Key Focus Areas: Testing GraphQL endpoints for introspection abuse, mass assignment, and denial-of-service via deep queries; checking RESTful API authentication enforcement.

Objective: To evaluate programmatic interfaces that lack a traditional graphical user interface (GUI) but present identical or enhanced threat exposures.


Information Gathering
Configuration and Deployment Management Testing
Identity Management Testing
Authentication Testing
Authorization Testing
Session Management Testing
Input Validation Testing
Error Handling
Cryptography
Business Logic Testing
Client-side Testing
API Testing